

Preparación de Entrevista — Arquitecto de Software

Antonio García Márquez · Proyecto de referencia: **mini-uber / driver-service**

1. Perfil profesional

Ingeniero de software con experiencia en el diseño y desarrollo de sistemas distribuidos basados en microservicios. Enfoque en arquitecturas escalables, resilientes y mantenibles, con especial atención a las buenas prácticas de diseño (SOLID, DDD, arquitectura hexagonal) y a la calidad del código (pruebas automatizadas, integración continua).

Puntos clave a comunicar

- Capacidad para diseñar sistemas desacoplados y orientados a eventos.
- Experiencia práctica llevando servicios desde el diseño hasta producción.
- Visión de arquitecto: equilibrio entre simplicidad, escalabilidad y coste de mantenimiento.
- Buenas prácticas de documentación y comunicación técnica con el equipo.

2. Arquitectura del proyecto de referencia: mini-uber

mini-uber es un sistema tipo "ride-hailing" (similar a Uber) compuesto por microservicios independientes que colaboran entre sí. **driver-service** es el microservicio responsable de la gestión de los conductores (alta, disponibilidad, ubicación, estado del vehículo, etc.) dentro de dicho ecosistema.

Responsabilidades de driver-service

- Registro y gestión del ciclo de vida de los conductores.
- Actualización y consulta del estado/disponibilidad en tiempo real.
- Exposición de una API para que otros servicios (por ejemplo, el servicio de viajes o de asignación) puedan consultar y reservar conductores disponibles.
- Publicación de eventos de dominio (ej. "conductor disponible", "conductor asignado") para mantener desacoplados a los servicios consumidores.

Decisiones arquitectónicas a defender

Aspecto	Decisión	Justificación
Estilo de arquitectura	Microservicios	Escalado independiente, despliegues autónomos y aislamiento de fallos por dominio.
Comunicación	Síncrona (REST/gRPC) para consultas, asíncrona (eventos) para notificaciones	Reduce el acoplamiento temporal entre servicios y mejora la resiliencia ante caídas puntuales.

Persistencia	Base de datos propia por servicio (database per service)	Evita acoplamiento a nivel de esquema y permite evolución independiente del modelo de datos.
Resiliencia	Circuit breaker / timeouts / reintentos en llamadas entre servicios	Evita el efecto cascada de fallos en un sistema distribuido.

3. Puntos fuertes a destacar

- **Arquitectura** Diseño de sistemas distribuidos desacoplados mediante eventos y APIs bien definidas (contract-first).
- **Calidad** Cultura de testing (unitario, integración, contract testing entre servicios) e integración continua.
- **Escalabilidad** Diseño pensado para el crecimiento horizontal: servicios sin estado, cacheo, particionado de datos.
- **Observabilidad** Logging estructurado, métricas y trazabilidad distribuida para depurar problemas entre microservicios.
- **Liderazgo técnico** Capacidad de guiar decisiones de arquitectura y mentorizar a otros desarrolladores.
- **Comunicación** Traducir decisiones técnicas complejas en términos comprensibles para perfiles no técnicos.

4. Preguntas y respuestas típicas de entrevista

4.1 Preguntas técnicas de arquitectura

¿Por qué elegirías una arquitectura de microservicios frente a un monolito?

Los microservicios permiten escalar y desplegar cada componente de forma independiente, aíslan fallos por dominio y facilitan que equipos distintos trabajen en paralelo. A cambio, aumentan la complejidad operativa (observabilidad, consistencia de datos, comunicación entre servicios), por lo que solo se justifican cuando el tamaño del equipo y del producto lo requiere. Un monolito bien modularizado puede ser preferible en etapas tempranas.

¿Cómo garantizas la consistencia de datos entre microservicios como driver-service y el servicio de viajes?

Evito transacciones distribuidas síncronas (2PC) y opto por consistencia eventual mediante el patrón Saga, coordinando el flujo con eventos de dominio (por ejemplo, "DriverAssigned", "TripCancelled") y compensaciones si un paso falla. Esto prioriza la disponibilidad del sistema sobre la consistencia inmediata.

¿Cómo evitarías que un fallo en un servicio downstream tumbe todo el sistema?

Aplicando patrones de resiliencia: timeouts agresivos, circuit breaker (ej. Resilience4j), reintentos con backoff exponencial, bulkheads para aislar recursos y respuestas degradadas (fallback) cuando sea posible, en lugar de propagar el fallo en cascada.

¿Cómo diseñarías la comunicación entre driver-service y el resto de servicios?

Para consultas puntuales de estado (¿está disponible este conductor?) usaría comunicación síncrona vía REST o gRPC con contratos versionados. Para notificar cambios de estado a múltiples consumidores (asignación, facturación, notificaciones push) usaría eventos asíncronos a través de un bus de mensajes (Kafka/RabbitMQ), desacoplando productores y consumidores en el tiempo.

¿Cómo gestionarías el versionado de la API de driver-service?

Versionado explícito (por ejemplo, en la URL o en cabeceras), manteniendo compatibilidad hacia atrás siempre que sea posible, evolucionando los contratos de forma aditiva y comunicando de forma clara las fechas de deprecación de versiones antiguas.

¿Qué estrategia seguirías para la base de datos de un servicio con alta carga de lectura, como la consulta de conductores disponibles?

Separar lecturas y escrituras (CQRS ligero), añadir una capa de caché (Redis) para las consultas más frecuentes, indexar adecuadamente por ubicación/estado y considerar particionado geográfico si el volumen lo requiere.

4.2 Escalabilidad y rendimiento

¿Cómo escalarías driver-service si el número de conductores activos se multiplicase por 100?

Primero mediría dónde está el cuello de botella (CPU, memoria, I/O de base de datos, red) con pruebas de carga. Después escalaría horizontalmente las instancias sin estado detrás de un balanceador, añadiría caché para las consultas de disponibilidad más frecuentes, optimizaría índices y consideraría particionar los datos por región geográfica si la consulta principal es "conductores disponibles cerca de X".

¿Qué diferencia hay entre escalado horizontal y vertical? ¿Cuándo usarías cada uno?

El escalado vertical añade más recursos (CPU/RAM) a una misma instancia; es simple pero tiene un límite físico y un único punto de fallo. El horizontal añade más instancias y reparte la carga; requiere que el servicio sea sin estado (o externalizar el estado) pero escala prácticamente sin límite y mejora la disponibilidad. En microservicios casi siempre se prioriza el escalado horizontal.

¿Cómo diseñarías el sistema de asignación de conductores para que sea eficiente a nivel geográfico (buscar el conductor más cercano)?

Usaría estructuras de datos e índices geoespaciales (geohash, quad-tree, o índices nativos como PostGIS/Redis GEO) en lugar de calcular distancias contra toda la tabla. Esto permite consultas de "conductores en un radio de X km" en tiempo casi constante en lugar de lineal con el número total de conductores.

¿Qué es el "hot partition" o punto caliente en un sistema distribuido y cómo lo evitarías?

Ocurre cuando una partición o clave concentra desproporcionadamente el tráfico (por ejemplo, todos los conductores de una ciudad muy grande). Se mitiga con un particionado más granular (por zonas más pequeñas), añadiendo un sufijo aleatorio a la clave (salting) o replicando/cacheando esa partición concreta.

¿Cómo medirías y mejorarías la latencia percibida por el usuario final al solicitar un conductor?

Instrumentaría el sistema con trazas distribuidas (OpenTelemetry) para ver el tiempo consumido en cada salto (API gateway, driver-service, matching, base de datos). Con esos datos priorizaría optimizar el camino crítico: cachear datos que no cambian con frecuencia, paralelizar llamadas independientes, y mover trabajo no esencial (notificaciones, analítica) fuera del camino síncrono mediante eventos.

4.3 Seguridad

¿Cómo asegurarías la comunicación y autenticación entre microservicios?

Usaría mTLS o un service mesh (Istio/Linkerd) para cifrar y autenticar el tráfico interno, tokens JWT de corta duración para la identidad del servicio/usuario, y un API gateway que centralice la autenticación de las peticiones externas antes de llegar a los servicios internos.

¿Cómo protegerías datos sensibles de los conductores (documentos, ubicación en tiempo real, datos bancarios)?

Cifrado en tránsito (TLS) y en reposo, minimización de datos (solo almacenar lo estrictamente necesario), tokenización o enmascaramiento de datos sensibles en logs, control de acceso basado en roles (RBAC) y auditoría de accesos. Para datos de pago, delegaría en un proveedor certificado PCI-DSS en lugar de almacenar tarjetas directamente.

¿Cómo evitarías que un servicio comprometido pueda acceder a todos los demás servicios del sistema?

Aplicando el principio de mínimo privilegio: cada servicio solo tiene permisos sobre los recursos que necesita, segmentación de red (network policies), autenticación mutua entre servicios y rotación periódica de credenciales/secrets gestionados con un vault (HashiCorp Vault, AWS Secrets Manager).

¿Qué es OWASP Top 10 y cómo lo tendrías en cuenta al diseñar una API?

Es una lista de los riesgos de seguridad más críticos en aplicaciones web (inyección, autenticación rota, exposición de datos sensibles, control de acceso roto, etc.). En el diseño de la API validaría y sanearía todas las entradas, usaría autenticación/autorización robustas, evitaría exponer información sensible en errores, y aplicaría rate limiting para prevenir abuso.

¿Cómo implementarías autenticación y autorización en driver-service usando OAuth2/JWT?

Delegaría la emisión de tokens a un Authorization Server (Keycloak, Auth0 o el propio API Gateway) siguiendo el flujo OAuth2 más adecuado (Authorization Code con PKCE para clientes móviles, Client Credentials para comunicación entre servicios). driver-service actuaría como resource server, validando el JWT (firma, expiración, audience) con Spring Security Resource Server, y aplicaría autorización de grano fino con roles/scopes o políticas basadas en atributos (ABAC) cuando el control por rol no sea suficiente (por ejemplo, un conductor solo puede ver y modificar su propio perfil).

¿Cómo evitarías ataques de inyección (SQL/NoSQL injection) en driver-service?

Usando siempre consultas parametrizadas o un ORM/Query DSL (JPA, jOOQ) en lugar de concatenar strings SQL, validando y saneando toda entrada del usuario con un esquema explícito (Bean Validation), aplicando el principio de mínimo privilegio en el usuario de base de datos, y ejecutando análisis estático (SAST) y escaneo de dependencias (SCA) en el pipeline de CI/CD para detectar patrones vulnerables antes de desplegar.

¿Cómo gestionarías el rate limiting y la protección contra ataques de denegación de servicio (DoS) en la API de conductores?

Aplicaría rate limiting en varias capas: en el API Gateway (por IP, por API key o por usuario autenticado) usando algoritmos como token bucket o sliding window, y a nivel de aplicación con librerías como Resilience4j o Bucket4j para proteger endpoints críticos (por ejemplo, la actualización de ubicación en tiempo real). Complementaría con un WAF y con autoscaling para absorber picos legítimos de tráfico sin penalizar a usuarios normales.

¿Cómo gestionarías los secretos (contraseñas de base de datos, API keys) de driver-service de forma segura?

Nunca en el código fuente ni en variables de entorno planas en el repositorio. Usaría un gestor de secretos dedicado (HashiCorp Vault, AWS Secrets Manager, Azure Key Vault) con inyección en tiempo de ejecución, rotación automática periódica, acceso auditado y cifrado en reposo, y escaneo de secretos (secret scanning) en el pipeline de CI para detectar fugas accidentales antes de hacer merge.

¿Qué diferencia hay entre autenticación y autorización, y cómo se refleja en el diseño de driver-service?

Autenticación responde a "¿quién eres?" (verificar identidad, por ejemplo con usuario/contraseña o un token JWT válido); autorización responde a "¿qué puedes hacer?" (permisos sobre un recurso concreto). En driver-service la autenticación se resuelve una vez en el borde del sistema (gateway) y se propaga como identidad verificada (JWT), mientras que la autorización se aplica en cada operación de negocio (por ejemplo, solo un administrador puede desactivar la cuenta de un conductor, mientras que el propio conductor solo puede actualizar sus datos).

¿Cómo diseñarías la seguridad para el envío de datos de geolocalización en tiempo real de los conductores?

Cifraría el canal completo con TLS (o wss:// si es WebSocket), autenticaría cada conexión con un token de corta duración renovado periódicamente, validaría que el conductor solo pueda enviar su propia ubicación (autorización a nivel de mensaje, no solo de conexión), y aplicaría límites de frecuencia de envío para evitar que un cliente comprometido inunde el sistema. Además, minimizaría la retención histórica de ubicaciones exactas por motivos de privacidad (GDPR), agregando o anonimizando los datos una vez cumplen su propósito operativo.

4.4 Patrones de diseño y DDD**¿Cómo aplicarías Domain-Driven Design (DDD) para definir los límites de driver-service frente a otros servicios (trips, billing, notifications)?**

Identificando los "bounded contexts": driver-service es dueño del ciclo de vida y estado del conductor; trips-service es dueño del ciclo de vida del viaje. La comunicación entre contextos se hace mediante un "context mapping" explícito (eventos o APIs), evitando que un servicio manipule directamente el modelo de datos de otro.

¿Qué es un agregado (aggregate) en DDD y cómo lo aplicarías al modelo de conductor?

Un agregado es un conjunto de entidades y value objects que se tratan como una unidad de consistencia transaccional, con una raíz (aggregate root) que controla el acceso. En driver-service, "Driver" sería la raíz del agregado, agrupando su perfil, estado de disponibilidad y ubicación actual, garantizando que cualquier cambio mantenga las invariantes de negocio (por ejemplo, no se puede asignar un viaje a un conductor no disponible).

¿Qué patrones usarías para desacoplar la lógica de negocio de los detalles de infraestructura (framework, base de datos)?

Arquitectura hexagonal (puertos y adaptadores) o Clean Architecture: el dominio define interfaces (puertos) y la infraestructura provee implementaciones (adaptadores). Esto permite testear la lógica de negocio sin depender de una base de datos real y facilita cambiar de tecnología sin tocar el núcleo del dominio.

¿Cuándo usarías CQRS y cuándo sería excesivo aplicarlo?

CQRS (separar modelos de lectura y escritura) tiene sentido cuando los patrones de lectura y escritura son muy distintos (por ejemplo, muchas más lecturas de "conductores disponibles" que escrituras de estado) o cuando se necesita escalar cada lado de forma independiente. Es excesivo para dominios simples con CRUD básico, donde añadiría complejidad sin un beneficio claro.

4.5 Testing y calidad**¿Qué estrategia de testing aplicarías en un ecosistema de microservicios?**

La pirámide de testing: muchas pruebas unitarias rápidas para la lógica de dominio, pruebas de integración para verificar la interacción con la base de datos o colas de mensajes, contract testing (ej. Pact) para validar que los contratos entre driver-service y sus consumidores no se rompen, y un número reducido de pruebas end-to-end para los flujos críticos.

¿Qué es el contract testing y por qué es especialmente útil en microservicios?

Permite verificar que un consumidor y un proveedor de una API cumplen un contrato acordado (esquema de petición/respuesta) sin necesidad de desplegar ambos servicios juntos. Evita regresiones cuando un equipo cambia su API sin coordinarse con los consumidores, algo muy común en sistemas con muchos servicios independientes.

¿Cómo probarías la resiliencia de driver-service ante fallos de sus dependencias?

Con pruebas de caos controladas (chaos engineering): simular latencia alta, caídas del servicio de base de datos o de la cola de mensajes, y verificar que los mecanismos de circuit breaker, timeouts y fallback se comportan como se espera, sin degradar el resto del sistema.

4.6 DevOps, CI/CD y despliegue**¿Cómo diseñarías el pipeline de CI/CD para driver-service?**

Build y pruebas unitarias/integración en cada commit, análisis estático y de seguridad (linters, SAST), construcción de una imagen inmutable (Docker), despliegue automático a un entorno de staging, pruebas de contrato/smoke tests, y despliegue a producción mediante una estrategia progresiva (canary o blue-green) con posibilidad de rollback automático si las métricas empeoran.

¿Qué diferencia hay entre un despliegue blue-green y uno canary?

Blue-green mantiene dos entornos idénticos (uno activo, uno en espera) y conmuta todo el tráfico de golpe a la nueva versión, permitiendo rollback instantáneo. Canary despliega la nueva versión a un pequeño porcentaje de tráfico real, se observan las métricas y se va incrementando gradualmente, reduciendo el radio de impacto si algo falla.

¿Cómo gestionarías la configuración y los secretos de driver-service en distintos entornos (dev, staging, producción)?

Externalizando la configuración del código (12-factor app), usando variables de entorno o un servicio de configuración centralizado, y gestionando los secretos con una herramienta dedicada (Vault, AWS Secrets Manager) en lugar de ficheros de configuración en el repositorio, con distintos valores/permisos por entorno.

4.7 Bases de datos y almacenamiento**¿Cómo diseñarías el modelo de datos de driver-service para soportar actualizaciones de ubicación muy frecuentes sin degradar el rendimiento?**

Separaría el dato de "ubicación en tiempo real" (alta frecuencia de escritura, se puede perder ocasionalmente) del dato "perfil del conductor" (baja frecuencia, debe ser consistente). La ubicación en tiempo real la guardaría en un almacén de baja latencia como Redis en lugar de la base de datos relacional principal, para no saturarla con escrituras constantes.

¿Cuándo elegirías una base de datos SQL frente a una NoSQL para un microservicio?

SQL cuando el dominio tiene relaciones complejas, necesita transacciones ACID fuertes y un esquema bien definido (por ejemplo, el perfil y estado del conductor). NoSQL (documental, clave-valor o series temporales) cuando el volumen de escritura es muy alto, el esquema es flexible o se necesita escalado horizontal nativo (por ejemplo, histórico de ubicaciones o eventos de telemetría).

¿Cómo migrarías el esquema de la base de datos de driver-service sin causar downtime?

Con migraciones "expand-contract": primero se añade la nueva estructura sin eliminar la antigua (expand), se despliega el código que escribe en ambas, se migran los datos existentes, se cambia la lectura a la nueva estructura, y solo cuando todo está validado se elimina la estructura antigua (contract). Esto evita romper compatibilidad durante despliegues graduales.

4.8 Mensajería y arquitectura orientada a eventos**¿Qué garantías de entrega ofrece un sistema de mensajería como Kafka y cómo afectan al diseño de driver-service?**

Normalmente "at-least-once": un mensaje puede procesarse más de una vez ante fallos o reintentos. Por eso los consumidores de eventos de driver-service deben ser idempotentes (por ejemplo, usando un identificador único de evento para no aplicar dos veces el mismo cambio de estado).

¿Qué es el patrón Outbox y por qué es relevante al publicar eventos desde driver-service?

Resuelve el problema de actualizar la base de datos y publicar un evento de forma atómica: en lugar de escribir en la base de datos y luego publicar al bus de mensajes (dos operaciones no atómicas que pueden desincronizarse), se escribe el evento en una tabla "outbox" dentro de la misma transacción, y un proceso aparte (o CDC) lo publica de forma fiable al bus de mensajes.

¿Cómo evitarías el acoplamiento excesivo entre servicios al diseñar los eventos de dominio?

Diseñando eventos que expresen "hechos de negocio" (ej. "DriverBecameAvailable") en lugar de replicar la estructura interna de la base de datos, versionando los esquemas de eventos de forma compatible hacia atrás, y evitando que los consumidores dependan de detalles de implementación internos del servicio emisor.

¿Cómo diseñarías el particionado de un topic de Kafka para eventos de driver-service (por ejemplo, "driver-location-updates")?

Usaría el ID del conductor como clave de partición (partition key), garantizando que todos los eventos de un mismo conductor se procesen en orden dentro de la misma partición. El número de particiones se dimensiona según el throughput objetivo y el paralelismo deseado en el consumo (cada partición solo puede ser leída por un consumidor de un mismo grupo a la vez), y se revisa que la clave elegida no genere "hot partitions" si hay conductores con actividad desproporcionada.

¿Qué diferencia hay entre las garantías "at-most-once", "at-least-once" y "exactly-once" en Kafka, y cuál usarías para eventos de ubicación de conductores?

"At-most-once" puede perder mensajes pero nunca duplica; "at-least-once" nunca pierde mensajes pero puede duplicarlos ante reintentos; "exactly-once" garantiza procesamiento único (con coste de rendimiento y complejidad, usando transacciones de Kafka o el patrón idempotent producer + consumidor idempotente). Para actualizaciones de ubicación en tiempo real, donde perder un evento puntual es tolerable porque llegará el siguiente enseguida, optaría por "at-least-once" con consumidores idempotentes en lugar de pagar el coste de "exactly-once".

¿Cómo gestionarías el orden de los mensajes en Kafka cuando necesitas escalar el consumo con varios consumidores en paralelo?

Kafka solo garantiza orden dentro de una misma partición, no entre particiones. Si el orden por conductor es crítico, uso el ID del conductor como clave para que sus eventos siempre caigan en la misma partición, y escalo el paralelismo aumentando el número de particiones y de consumidores dentro del grupo (uno por partición como máximo), aceptando que el orden global entre distintos conductores no está garantizado ni es necesario para el caso de uso.

¿Qué es un "consumer group" en Kafka y cómo afecta al diseño de driver-service?

Un consumer group es un conjunto de consumidores que se reparten las particiones de un topic para procesar mensajes en paralelo sin duplicar trabajo entre ellos; cada mensaje de una partición lo procesa solo un consumidor del grupo. En driver-service escalaría horizontalmente las instancias que procesan eventos de asignación de viajes añadiéndolas al mismo consumer group, y usaría grupos distintos cuando varios servicios (billing, notifications) necesiten consumir de forma independiente el mismo topic.

¿Cómo manejarías un mensaje "envenenado" (poison message) que hace fallar repetidamente a un consumidor de Kafka?

Configuraría un número máximo de reintentos con backoff y, si se agotan, enviaría el mensaje a una dead-letter queue (DLQ o topic de errores) en lugar de bloquear indefinidamente el procesamiento de la partición (lo cual pararía también los mensajes correctos posteriores). Desde la DLQ se puede alertar, inspeccionar manualmente y reprocesar una vez corregida la causa, sin perder el mensaje original ni degradar el resto del sistema.

¿Cuándo usarías Kafka Streams o una tecnología de stream processing frente a consumidores tradicionales de Kafka?

Un consumidor tradicional procesa mensajes de forma independiente; Kafka Streams (o Flink) aporta procesamiento con estado, ventanas temporales, agregaciones y joins entre streams. Lo usaría, por ejemplo, para calcular en tiempo real métricas como "número de conductores disponibles por zona en los últimos 5 minutos" a partir del stream de eventos de ubicación, algo que sería complejo y propenso a errores de implementar manualmente con consumidores simples gestionando el estado a mano.

¿Cómo evolucionarías el esquema de un evento de Kafka sin romper a los consumidores existentes?

Usaría un registro de esquemas (Schema Registry) con Avro o Protobuf, aplicando compatibilidad "backward" (los consumidores antiguos pueden leer eventos nuevos) como mínimo: añadiendo campos opcionales con valores por defecto en lugar de eliminar o renombrar campos existentes, y versionando de forma explícita cuando un cambio incompatible sea inevitable, permitiendo que ambas versiones convivan durante una migración gradual.

4.9 Observabilidad y monitorización

¿Qué métricas clave monitorizarías en driver-service en producción?

Latencia y tasa de error de la API (p50/p95/p99), tasa de conductores disponibles frente a la demanda, tiempo de procesamiento de eventos, saturación de la base de datos/caché, y métricas de negocio como tiempo medio hasta encontrar un conductor disponible.

¿Cómo depurarías un problema intermitente de latencia que solo aparece en producción y afecta a varios servicios?

Usando trazas distribuidas (OpenTelemetry/Jaeger) para seguir una petición a través de todos los servicios implicados e identificar en qué salto concreto se produce la latencia, correlacionando con métricas de infraestructura (CPU, memoria, I/O) y logs estructurados con un identificador de correlación común.

¿Qué diferencia hay entre logs, métricas y trazas, y cómo se complementan?

Los logs registran eventos discretos con contexto detallado; las métricas son series numéricas agregadas en el tiempo (ideales para alertas y dashboards); las trazas muestran el recorrido completo de una petición a través de múltiples servicios. Juntas forman los "tres pilares de la observabilidad" y permiten pasar de "algo va mal" (métrica/alerta) a "por qué va mal" (traza) a "qué pasó exactamente" (log).

4.10 Cloud e infraestructura

¿Cómo desplegarías driver-service en un entorno cloud con alta disponibilidad?

Contenedores orquestados con Kubernetes, desplegados en al menos dos zonas de disponibilidad, con auto-scaling horizontal basado en métricas de carga, health checks/liveness y readiness probes, y una base de datos gestionada con réplicas para tolerancia a fallos.

¿Qué ventajas y riesgos tiene usar servicios gestionados (managed services) de un proveedor cloud frente a autogestionar la infraestructura?

Ventajas: menor carga operativa, alta disponibilidad y parcheo gestionados por el proveedor, mayor velocidad de entrega. Riesgos: vendor lock-in, menor control sobre configuraciones específicas y coste potencialmente mayor a gran escala. La decisión depende del tamaño del equipo de plataforma y de cuánto valor aporta controlar ese detalle frente a delegarlo.

4.11 Preguntas de comportamiento y experiencia

Cuéntame sobre un momento en el que tuviste que tomar una decisión de arquitectura difícil.

Estructura la respuesta con el método STAR: Situación (contexto del proyecto), Tarea (el problema concreto a resolver), Acción (las alternativas evaluadas y por qué elegiste una) y Resultado (impacto medible: rendimiento, mantenibilidad, reducción de incidencias). Ejemplo: decidir entre acoplar dos servicios vía llamada síncrona o desacoplarlos con eventos, explicando el trade-off de latencia frente a resiliencia.

¿Cómo gestionas el desacuerdo técnico dentro de un equipo?

Fomento la discusión basada en datos y no en opiniones: pruebas de concepto, benchmarks o documentos de diseño (ADR - Architecture Decision Records) que comparen alternativas. Busco el consenso, pero si no se alcanza, se toma una decisión con un responsable claro, documentando el porqué para poder revisarla más adelante si las circunstancias cambian.

Describe una vez que tuviste que aprender una tecnología nueva rápidamente.

Explica tu proceso de aprendizaje (documentación oficial, prototipos pequeños, pair programming) y cómo lo aplicaste de forma segura en el proyecto real, por ejemplo introduciendo la tecnología primero en un componente de bajo riesgo antes de generalizarla.

¿Cómo priorizas entre entregar rápido y mantener la calidad/arquitectura limpia?

Reconozco la deuda técnica como una herramienta válida cuando se gestiona de forma consciente: documento la decisión, su coste futuro y planifico cuándo abordarla. La calidad no es negociable en aspectos críticos (seguridad, integridad de datos), pero sí puede haber margen en aspectos no críticos a corto plazo.

¿Cómo comunicas decisiones de arquitectura a perfiles no técnicos (negocio, producto)?

Traduzco el impacto técnico a términos de negocio: tiempo, coste, riesgo y beneficio. Evito la jerga y uso analogías o diagramas simples, centrándome en qué gana el negocio (fiabilidad, velocidad de entrega futura) en lugar de en el detalle de implementación.

Cuéntame sobre un fallo grave en producción del que fueras responsable. ¿Qué pasó y qué aprendiste?

Usa STAR y sé honesto: describe el incidente, el impacto real (usuarios/tiempo de caída), la causa raíz identificada mediante un post-mortem sin buscar culpables, y las acciones concretas implementadas después (nuevas alertas, tests, cambios de proceso) para que no vuelva a ocurrir. Los entrevistadores valoran la capacidad de aprender y mejorar procesos, no la ausencia de errores.

¿Cómo convences a un equipo o a negocio de invertir tiempo en pagar deuda técnica en lugar de construir nuevas funcionalidades?

Cuantifico el impacto de la deuda en términos que negocio entienda: incidencias recurrentes, tiempo perdido en despliegues, riesgo de un fallo mayor, o ralentización de la velocidad de entrega futura. Propongo abordarla de forma incremental, integrada en el propio desarrollo de features, en lugar de pedir un "parón" completo para refactorizar.

Describe cómo mentorizas o ayudas a crecer a desarrolladores menos senior en tu equipo.

Combino code reviews constructivas (explicando el porqué, no solo el qué), pair/mob programming en tareas complejas, y fomento la autonomía dándoles problemas con margen para proponer su propia solución antes de intervenir. También comparto documentación y ADRs para que entiendan el contexto de las decisiones existentes.

¿Cómo manejas la presión de plazos ajustados sin comprometer la estabilidad del sistema?

Negocio el alcance antes que la calidad: identifico con el equipo de producto qué es imprescindible y qué se puede posponer, comunico riesgos de forma temprana y transparente, y reservo tiempo para pruebas de los flujos críticos aunque se reduzca el de funcionalidades secundarias.

4.12 Kafka y streaming de eventos en profundidad**¿Por qué elegirías Kafka frente a un broker tradicional como RabbitMQ para el flujo de eventos de driver-service?**

Kafka está optimizado para alto throughput y retención duradera del log de eventos (permite "replay" a nuevos consumidores o para recuperación), con particionado nativo para escalar el consumo en paralelo. RabbitMQ brilla en enrutamiento flexible de mensajes (exchanges, colas por prioridad) y en casos de baja latencia con colas más pequeñas. Para eventos de dominio de alto volumen (ubicaciones, estados de conductor) que varios servicios necesitan consumir de forma independiente y potencialmente reprocesar, Kafka encaja mejor.

¿Cómo diseñarías el procesamiento de streams para calcular en tiempo real la demanda y oferta de conductores por zona geográfica?

Usaría Kafka Streams (o Flink) con ventanas temporales deslizantes (sliding windows) agregando el stream de eventos de ubicación y estado por geohash o celda de rejilla, manteniendo el estado agregado en un state store local respaldado por un "changelog topic" para tolerancia a fallos. El resultado (oferta/demanda por zona) se publicaría en un topic compactado o se expondría vía una API de consulta (KTable) para que otros servicios lo consuman sin recalcularlo.

¿Qué es el "consumer lag" en Kafka y cómo lo monitorizarías y actuarías sobre él?

Es la diferencia entre el offset más reciente publicado en una partición y el offset que el consumidor ha procesado; indica si el consumidor va "atrasado" respecto a la producción de eventos. Lo monitorizaría con métricas expuestas por el propio broker o el cliente (JMX/Micrometer) y alertaría si supera un umbral, ya que puede indicar que el consumidor es demasiado lento (hay que escalar horizontalmente añadiendo particiones y consumidores) o que hay un problema de rendimiento en el procesamiento.

¿Cómo asegurarías que un evento de dominio se publica de forma fiable incluso si Kafka no está disponible momentáneamente?

Con el patrón Outbox: el evento se escribe en la misma transacción de base de datos que el cambio de estado, y un proceso de publicación (poller o Debezium con Change Data Capture) lee la tabla outbox y reintenta publicarlo a Kafka hasta conseguirlo, marcándolo como enviado. Esto evita el escenario en el que la base de datos se actualiza pero el evento se pierde porque Kafka estaba caído en ese instante.

¿Cómo probarías (testing) la lógica de un consumidor de Kafka sin depender de un clúster real?

Para pruebas unitarias, aislaría la lógica de negocio del framework de mensajería (puertos y adaptadores) y la testearía con mocks simples. Para pruebas de integración usaría Testcontainers levantando un broker Kafka real en un contenedor Docker efímero, verificando la serialización/deserialización, el particionado y el comportamiento ante fallos (reintentos, DLQ) en condiciones muy cercanas a producción.

4.13 Programación reactiva y WebClient

¿Qué diferencia hay entre programación reactiva (Reactor/WebFlux) y el modelo tradicional bloqueante de Spring MVC, y cuándo elegirías cada uno?

Spring MVC usa un hilo por petición que se bloquea mientras espera I/O (base de datos, llamadas a otros servicios); WebFlux usa un modelo no bloqueante basado en Reactor (Mono/ Flux) sobre un pool reducido de hilos que nunca se bloquean, escalando mejor bajo alta concurrencia con I/O intensivo (muchas conexiones simultáneas, streaming de datos). Elegiría WebFlux para el flujo de ubicaciones en tiempo real de miles de conductores concurrentes, y Spring MVC (más simple de razonar y depurar) para CRUDs de negocio con carga moderada.

¿Cómo usarías WebClient para llamar a otros microservicios (por ejemplo, el servicio de viajes) de forma no bloqueante?

Configuraría un WebClient con un connection pool dimensionado, timeouts explícitos de conexión y respuesta, y lo combinaría con operadores de Reactor como `timeout` `retryWhen` con backoff exponencial y `onErrorResume` para fallback, evitando bloquear el hilo con `.block()` salvo en el borde de una capa síncrona (por ejemplo, un controlador MVC que necesita el resultado antes de responder), donde su uso está justificado y controlado.

¿Qué problema resuelve el "backpressure" en programación reactiva y cómo se aplicaría en el stream de ubicaciones de conductores?

El backpressure evita que un productor rápido sature a un consumidor más lento, permitiendo que el propio consumidor indique cuánta información puede procesar (en lugar de que el productor empuje datos sin control, agotando memoria). En el stream de ubicaciones, si el servicio de matching no puede procesar todos los eventos al ritmo en que llegan, aplicaría estrategias como buffer con límite, "drop" de eventos antiguos (solo importa la última posición) o muestreo (sample) para mantener solo la posición más reciente relevante en lugar de encolar indefinidamente.

¿Qué errores comunes se cometen al migrar código bloqueante a WebFlux y cómo los evitarías?

El error más habitual es mezclar código bloqueante (JDBC clásico, llamadas bloqueantes a otros sistemas) dentro del pipeline reactivo sin aislarlo, lo que bloquea los pocos hilos del event loop y anula la ventaja de WebFlux. Lo evitaría usando drivers reactivos nativos (R2DBC en vez de JPA/JDBC) o, si no es posible, delegando las llamadas bloqueantes a un "bounded elastic scheduler" dedicado para no afectar al resto del sistema.

¿Cómo depurarías y testearías código reactivo, dado que las trazas de pila (stack traces) suelen ser difíciles de interpretar?

Activaría "Reactor Debug Agent" o `Hooks.onOperatorDebug()` en entornos de desarrollo/test para obtener trazas enriquecidas con el punto real de ensamblado del pipeline, usaría `StepVerifier` de reactor-test para verificar de forma determinista las señales emitidas (`onNext`, `onError`, `onComplete`) en pruebas unitarias, y me apoyaría en trazas distribuidas (contexto de Reactor propagado a través de operadores) para seguir una petición reactiva de extremo a extremo en producción.

4.14 Concurrencia: hilos y Virtual Threads

¿Qué son los Virtual Threads de Project Loom (Java 21) y qué problema resuelven frente a los hilos de plataforma tradicionales?

Son hilos ligeros gestionados por la JVM (no directamente por el sistema operativo) que se multiplexan sobre un pequeño número de hilos de plataforma (carrier threads). Permiten crear millones de hilos con un coste de memoria y creación mucho menor que los hilos tradicionales, y cuando un virtual thread se bloquea en I/O, la JVM libera automáticamente el carrier thread para que ejecute otro virtual thread, en lugar de dejarlo ocioso. Esto permite escribir código bloqueante "de toda la vida" (más simple de leer y depurar que el reactivo) y obtener una escalabilidad similar a la de un modelo no bloqueante.

¿Cuándo elegirías Virtual Threads frente a programación reactiva (WebFlux) para driver-service?

Virtual Threads son atractivos cuando el cuello de botella es el número de conexiones concurrentes esperando I/O (muchas llamadas a base de datos o a otros servicios) y se quiere mantener la simplicidad del modelo de programación imperativo/bloqueante, evitando la curva de aprendizaje y la complejidad de depuración de Reactor. Optaría por WebFlux cuando ya se necesitan operadores reactivos avanzados (backpressure, combinación de streams, procesamiento de eventos continuos) o cuando el ecosistema ya está construido sobre drivers reactivos (R2DBC). Para muchos servicios CRUD con alta concurrencia de E/S, Virtual Threads ofrecen casi el mismo beneficio de escalabilidad con mucho menos código.

¿Qué limitaciones o "pinning" tienen los Virtual Threads que un arquitecto debe conocer antes de adoptarlos en producción?

Un virtual thread puede quedar "clavado" (pinned) a su carrier thread —sin poder liberarlo durante un bloqueo— cuando ejecuta código dentro de un bloque `synchronized` o dentro de métodos nativos que bloquean, lo que puede degradar la escalabilidad si se abusa de bloques sincronizados con I/O dentro. La recomendación es sustituir `synchronized` por `ReentrantLock` en secciones críticas de alta concurrencia, y validar con las herramientas de diagnóstico de la JVM (JFR) cuánto "pinning" ocurre realmente antes de migrar código sensible a Virtual Threads.

¿Qué diferencia hay entre concurrencia y paralelismo, y cómo se aplica al procesamiento de eventos de driver-service?

Concurrencia es la capacidad de gestionar múltiples tareas que progresan de forma intercalada en el tiempo (no necesariamente al mismo instante); paralelismo es ejecutar varias tareas literalmente al mismo tiempo en distintos núcleos de CPU. El procesamiento de eventos de Kafka es concurrente por naturaleza (muchos consumidores gestionando muchas particiones), y se vuelve paralelo cuando se ejecutan en distintos hilos/núcleos simultáneamente; un cálculo intensivo en CPU (por ejemplo, recalcular rutas óptimas) se beneficia de paralelismo real, mientras que esperar respuestas de red se beneficia principalmente de concurrencia (no bloquear hilos esperando).

¿Cómo evitarías condiciones de carrera (race conditions) al actualizar de forma concurrente el estado de disponibilidad de un conductor?

A nivel de base de datos, usaría bloqueo optimista (una columna de versión que falla la actualización si el registro cambió desde que se leyó) en lugar de bloqueo pesimista para no penalizar el rendimiento en el caso común sin conflicto. A nivel de aplicación, evitaría compartir estado mutable entre hilos y, si fuera imprescindible, usaría estructuras thread-safe (`AtomicReference` , colecciones concurrentes) o serializaría las actualizaciones críticas de un mismo conductor a través de una única partición de Kafka o una clave de bloqueo distribuido (Redis) para garantizar que solo una actualización se procese a la vez por conductor.

4.15 Sistemas en tiempo real y baja latencia

¿Qué opciones de comunicación en tiempo real usarías para notificar a un conductor que se le ha asignado un viaje, y cómo elegirías entre ellas?

Para push bidireccional de baja latencia, WebSocket (o STOMP sobre WebSocket) si el cliente mantiene una conexión persistente; para notificaciones a apps móviles en segundo plano, un servicio de push nativo (FCM/APNs) es más eficiente energéticamente que mantener un socket abierto constantemente. Para actualizaciones unidireccionales del servidor al cliente sin necesidad de baja latencia extrema, Server-Sent Events (SSE) es más simple que WebSocket. La elección depende de si la comunicación es bidireccional, de la frecuencia de los eventos y del consumo de batería/recursos del cliente.

¿Cómo escalarías horizontalmente un servicio de WebSockets que mantiene miles de conexiones persistentes con conductores?

Cada instancia mantiene un subconjunto de conexiones activas, por lo que necesito un mecanismo para enrutar un mensaje al servidor correcto: un "sticky routing" en el balanceador basado en el ID de conexión, y un backplane compartido (Redis Pub/Sub o un topic de Kafka) para que cualquier instancia pueda publicar un evento que llegue a la instancia que realmente tiene la conexión abierta con ese conductor concreto. También monitorizaría el número de conexiones por instancia para dimensionar el autoscaling.

¿Qué trade-offs consideras entre latencia, throughput y consistencia al diseñar el envío de la ubicación de un conductor cada pocos segundos?

Enviar cada actualización individualmente minimiza la latencia pero aumenta el número de mensajes (throughput) y la carga en el sistema; agrupar varias actualizaciones en lotes (batching) reduce la sobrecarga por mensaje pero introduce latencia adicional. Para este caso de uso, priorizaría baja latencia (el sistema necesita saber la posición reciente para el matching) y aceptaría cierta pérdida de precisión histórica: solo me interesa la última posición conocida, por lo que puedo aplicar compactación (quedarme solo con el último valor por clave) en el propio topic de Kafka en lugar de conservar todo el histórico.

¿Cómo garantizarías baja latencia extremo a extremo en el flujo "solicitud de viaje → asignación de conductor"?

Evitando saltos síncronos innecesarios en el camino crítico, cacheando datos que no cambian con frecuencia (perfil del conductor) para no consultarlos en cada petición, usando índices geoespaciales para encontrar candidatos cercanos en tiempo casi constante, procesando en paralelo pasos independientes (verificar disponibilidad y calcular tarifa estimada), y midiendo continuamente con trazas distribuidas para detectar regresiones de latencia antes de que impacten a los usuarios.

¿Qué es un "circuit breaker" y cómo protegería un flujo de tiempo real ante la caída de un servicio dependiente (por ejemplo, el cálculo de tarifas)?

Un circuit breaker monitoriza la tasa de fallos de las llamadas a un servicio dependiente y, al superar un umbral, "abre el circuito" dejando de intentar la llamada durante un tiempo (fallando rápido en su lugar) para no acumular latencia ni agotar recursos esperando timeouts repetidos; tras un periodo, permite pasar algunas peticiones de prueba ("half-open") para comprobar si el servicio se ha recuperado. En el flujo de asignación, si el servicio de tarifas falla, devolvería una estimación cacheada o un valor por defecto en lugar de bloquear la asignación del conductor, priorizando la disponibilidad del flujo crítico.

5. Preguntas para hacer al entrevistador

- ¿Cómo está estructurado el equipo de arquitectura y qué grado de autonomía tienen los equipos de producto sobre sus propios microservicios?
- ¿Qué herramientas de observabilidad (logging, métricas, trazas) utilizan actualmente para monitorizar el sistema en producción?
- ¿Cómo gestionan actualmente la consistencia de datos entre servicios? ¿Usan algún patrón como Saga o eventos de dominio?
- ¿Cuál es el mayor reto de arquitectura al que se han enfrentado en el último año?
- ¿Qué proceso siguen para tomar y documentar decisiones de arquitectura (ADRs, RFCs, etc.)?
- ¿Cómo es el proceso de despliegue y cuál es la frecuencia de entrega a producción?

Documento de preparación personal · generado como material de apoyo para la entrevista técnica.